UNIVERSIDAD NACIONAL AUTÓNOMA DE MÉXICO
FACULTAD DE INGENIERÍA

DIVISIÓN DE INGENIERÍA ELÉCTRICA
COORDINACIÓN DE COMPUTACIÓN

BASES DE DATOS

Tarea 9

Propiedades ACID y Niveles de Aislamiento

Nombre del alumno: Yukioayax Canek Gabriel Hernández
Profesor: Fernando Arreola Franco
Grupo: 1
Semestre en curso: 2026-2

Instrucciones

Investigar:

- Propiedades ACID en bases de datos relacionales.
- Niveles de aislamiento en bases de datos relacionales.

Propiedades ACID

Las propiedades ACID (Atomicidad, Consistencia, Aislamiento, Durabilidad) son un conjunto de garantías que aseguran la integridad y fiabilidad de las transacciones en un sistema de bases de datos relacionales, fueron formalizadas por Jim Gray a finales de los años 70 y son implementadas por los sistemas de gestión de bases de datos (SGBD) mediante mecanismos como el registro de transacciones (log), bloqueos y control de concurrencia.

Atomicidad (Atomicity)

"Todo o nada". Una transacción es una unidad indivisible de trabajo: o bien se ejecutan todas sus operaciones, o no se ejecuta ninguna, si alguna parte de la transacción falla, el sistema debe deshacer (rollback) cualquier cambio realizado por las operaciones previas, dejando la base de datos en el estado anterior a la transacción.
Ejemplo: Transferencia bancaria entre dos cuentas:

```
BEGIN TRANSACTION;
    UPDATE Cuenta SET saldo = saldo - 500 WHERE id = 'A';
    UPDATE Cuenta SET saldo = saldo + 500 WHERE id = 'B';
COMMIT;
```

Si la segunda actualización falla (por ejemplo, cuenta B inexistente), la atomicidad garantiza que el débito de la cuenta A se revierta completamente.

Consistencia (Consistency)

"Integridad preservada". Una transacción debe llevar la base de datos de un estado válido a otro estado válido, respetando todas las restricciones definidas (claves primarias, claves foráneas, restricciones `CHECK`, reglas de negocio), la consistencia es responsabilidad compartida entre el SGBD (que hace cumplir las restricciones declarativas) y el programador (que asegura la lógica de negocio correcta).
Ejemplo: Una restricción que exige que el saldo nunca sea negativo, si una transacción intenta dejar un saldo negativo, el SGBD la rechazará al validar la restricción, manteniendo la consistencia.

Aislamiento (Isolation)

"Ejecución concurrente como si fuera en serie". Las transacciones que se ejecutan concurrentemente deben aislarse unas de otras, de modo que los efectos intermedios de una transacción no sean visibles para otras hasta que se complete (commit), el nivel de aislamiento define el grado de interferencia permitido; un aislamiento completo (serializable) evita las llamadas anomalías de concurrencia.
Ejemplo: Dos usuarios intentan comprar el último asiento de un vuelo, con un aislamiento adecuado, solo una transacción podrá reservar el asiento; la otra verá que ya no está disponible o será bloqueada hasta que la primera finalice.

Durabilidad (Durability)

"Persistencia garantizada". Una vez que una transacción ha sido confirmada (`COMMIT`), sus cambios deben permanecer en la base de datos de forma permanente, incluso en caso de fallo del sistema (corte de energía, caída del servidor), esto se logra escribiendo los cambios en un registro de transacciones (log) en almacenamiento no volátil antes de confirmar la transacción (write-ahead logging).
Ejemplo: Después de que el banco confirma una transferencia, el usuario puede ver el nuevo saldo reflejado incluso si el servidor se reinicia inmediatamente después.

Niveles de aislamiento en SQL

El estándar SQL define cuatro niveles de aislamiento que controlan la visibilidad de los cambios entre transacciones concurrentes. Cada nivel evita ciertas *anomalías de concurrencia*:

- **Lectura sucia (Dirty Read):** Una transacción lee datos escritos por otra transacción aún no confirmada (y que podría revertirse).
- **Lectura no repetible (Non-repeatable Read):** Una transacción lee dos veces el mismo registro y obtiene valores diferentes porque otra transacción lo modificó y confirmó entre ambas lecturas.
- **Lectura fantasma (Phantom Read):** Una transacción ejecuta dos veces la misma consulta con una condición de rango y obtiene un número diferente de filas porque otra transacción insertó o eliminó filas que satisfacen dicha condición.

Los cuatro niveles de aislamiento, ordenados del menor al mayor aislamiento, son:

READ UNCOMMITTED

Es el nivel más bajo, permite **lecturas sucias**, lecturas no repetibles y lecturas fantasma, prácticamente no ofrece garantías de aislamiento y rara vez se usa en aplicaciones que requieren integridad.
Ejemplo en PostgreSQL:

```
BEGIN TRANSACTION ISOLATION LEVEL READ UNCOMMITTED;
-- (PostgreSQL trata READ UNCOMMITTED como READ COMMITTED)
```

READ COMMITTED

Nivel predeterminado en la mayoría de SGBD (incluido PostgreSQL), **evita lecturas sucias**, pero permite lecturas no repetibles y fantasma, cada consulta dentro de la transacción ve solo los datos confirmados al inicio de *esa consulta*, no al inicio de la transacción.
Ejemplo:

```
-- Sesión 1
BEGIN;
UPDATE empleados SET salario = salario * 1.1 WHERE id = 100;
-- (no se hace COMMIT aún)

-- Sesión 2 (READ COMMITTED)
BEGIN;
SELECT salario FROM empleados WHERE id = 100;  -- ve el valor antiguo
-- Sesión 1 hace COMMIT;
SELECT salario FROM empleados WHERE id = 100;  -- ahora ve el nuevo valor (lectura no re
COMMIT;
```

REPEATABLE READ

Garantiza que si una transacción lee un registro dos veces, obtendrá el mismo valor en ambas lecturas (**evita lecturas sucias y no repetibles**), sin embargo, aún pueden ocurrir **lecturas fantasma** (inserciones en rangos). En PostgreSQL, este nivel **también evita las lecturas fantasma** gracias a su implementación basada en instantáneas (snapshot isolation).
Ejemplo en PostgreSQL:

```
BEGIN TRANSACTION ISOLATION LEVEL REPEATABLE READ;
SELECT COUNT(*) FROM pedidos WHERE fecha = '2026-04-13'; -- digamos 10
-- Otra transacción inserta un pedido con fecha '2026-04-13' y hace COMMIT
SELECT COUNT(*) FROM pedidos WHERE fecha = '2026-04-13'; -- sigue siendo 10 (sin fantasm
COMMIT;
```

SERIALIZABLE

El nivel más estricto. **Evita todas las anomalías**: lecturas sucias, no repetibles y fantasma, las transacciones concurrentes se ejecutan de manera que el resultado final sea equivalente a alguna ejecución en serie de las mismas. Puede provocar más bloqueos y menor rendimiento, pero es necesario cuando la consistencia absoluta es crítica.
Ejemplo: Si dos transacciones intentan modificar datos que se solapan de forma incompatible, una de ellas será rechazada con un error de serialización y deberá reintentarse.

```
BEGIN TRANSACTION ISOLATION LEVEL SERIALIZABLE;
-- operaciones críticas...
COMMIT;
```

Cuadro comparativo de niveles de aislamiento

Nivel de aislamiento	Lectura sucia	Lectura no repetible	Lectura fantasma
READ UNCOMMITTED	Posible	Posible	Posible
READ COMMITTED	Evitada	Posible	Posible
REPEATABLE READ	Evitada	Evitada	Posible (excepto en PG)
SERIALIZABLE	Evitada	Evitada	Evitada

Cuadro 1: Anomalías permitidas según el nivel de aislamiento estándar.

Conclusión

Las propiedades ACID constituyen la base de la fiabilidad en los sistemas transaccionales, garantizando que las operaciones complejas se completen de manera segura incluso ante fallos y accesos concurrentes, los niveles de aislamiento ofrecen un compromiso entre consistencia estricta y rendimiento, permitiendo a los desarrolladores elegir la protección adecuada según los requisitos de la aplicación. Comprender estos conceptos es esencial para diseñar aplicaciones robustas que interactúen con bases de datos relacionales.

Referencias

[1] J. Gray, "The Transaction Concept: Virtues and Limitations," en *Proc. 7th Int. Conf. Very Large Data Bases*, Cannes, France, 1981, pp. 144-154.

[2] T. Härder y A. Reuter, "Principles of Transaction-Oriented Database Recovery," *ACM Computing Surveys*, vol. 15, no. 4, pp. 287-317, Dic. 1983.

[3] The PostgreSQL Global Development Group, *PostgreSQL 16 Documentation: Chapter 13. Concurrency Control*, [En línea]. Disponible en: `https://www.postgresql.org/docs/16/transaction-iso.html`.

[4] A. Silberschatz, H. F. Korth, y S. Sudarshan, *Database System Concepts*, 7th ed. New York, NY, USA: McGraw-Hill, 2020, pp. 627-676.

[5] R. Elmasri y S. B. Navathe, *Fundamentals of Database Systems*, 7th ed. Boston, MA, USA: Pearson, 2016, pp. 761-796.